Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Voxelitization Problem with Divide & Conquer

Disusun oleh:

Stanislaus Ardy Bramantyo 18223057

Muhammad Azzam Robbani 18223025

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

Laporan Tugas Kecil 2 ..... **1**
Permasalahan **3**
Pendekatan Solusi **3**
Implementasi Solusi **5**
1. Data: Vector3D & Triangle 5
a. Vector3D.java 5
b. Triangle.java 6
2. Input: Membaca.obj 6
3. Node Kubus: OctreeNode.java 7
4. SAT: IntersectionTest.java 8
5. Divide & Conquer: Octree.java 8
6. Output: ObjectWriter.java 9
Analisis Kompleksitas **28**
A. Parsing 28
B. SAT Intersection Test 28
C. Divide & Conquer Leaf Node 28
D. Output 29
Pengujian Test Case **29**
A. Pengujian 1: cow.obj 29
B. Pengujian 2: line.obj 30
C. Pengujian 3: pumpkin.obj 32
D. Pengujian 4: teapot.obj 33
E. Pengujian 5: teddybear.obj 35
Repository **37**
Pernyataan Kejujuran **37**
Lampiran **37**

Permasalahan

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

Pendekatan Solusi

Permasalahan *voxelization* yang telah diajukan akan memanfaatkan solusi yang menggunakan pendekatan **Divide and Conquer** sebagai solusinya. Pendekatan tersebut akan memanfaatkan struktur data Octree (menyesuaikan juga dengan permasalahan yang ada). Secara ide, dapat digambarkan sebagai ruang 3D yang nge-*wrap* seluruh objek yang dipecah terus-menerus menjadi 8 bagian yang lebih kecil setiap "tahap"nya. Pemecahan ini akan tetap dikontrol oleh *user* melalui *input* pada variabel `maxDepth`, dimana akan menentukan seberapa dalam (*depth*) potongan tersebut dilakukan.

Sebelum Octree dibangun, program akan lebih fokus ke tahap pembacaan file `.obj` yang berisikan informasi mengenai objek 3D berbentuk *Triangle Mesh* atau sekumpulan segitiga. Dari seluruh *vertex* yang ada di dalam objek tersebut, akan dicari nilai `min-max` tiap sumbunya untuk membentuk suatu kubus besar yang disebut `Bounding Box` yang akan *wrap* objek tersebut (Konsep dari Axis-Aligned Bounding Box). Kubus tersebut menjadi titik awal dari Octree yang akan nanti dipecah menjadi 8 oktan yang terus dipecah lagi menjadi 8 hingga `maxDepth`.

Terdapat usaha efisiensi mengenai proses ini, yaitu bagian *pruning*. Ternyata, tidak semua oktan hasil suatu pecahan harus ditelusuri hingga "ujung". Kita dapat memprioritaskan *resource* untuk menelusuri oktan-oktan tertentu, seperti yang bersentuhan dengan permukaan objeknya saja. Untuk mengeceknya, kami memanfaatkan algoritma **SAT** (Separating Axis Theorem), sebuah algoritma deteksi *intersect* antara Kubus & Segitiga. Kalau tidak ada satupun segitiga yang berpotongan dengan sebuah oktan, oktan itu dapat di-*pruning*.

Node-node yang setelah dieksplorasi dan mencapai `maxDepth` dan ternyata masih *intersect* dengan permukaan objek dapat dinamakan ***leaf node***, yaitu *voxel* dari hasil akhir. Tiap-tiap *leaf node* yang ada akan dikonversi menjadi sebuah kubus kecil dengan 8 *vertex*

dan 12 segitiga, lalu akan ditulis ke sebuah *file* `.obj` untuk nanti dapat di-*render* 3Dnya. Hasil yang diharapkan akan memiliki bentuk yang persis dengan perbedaan permukaan berbentuk balok-balok, seperti di Minecraft.

Implementasi Solusi

Penjelasan pada bagian Implementasi Solusi hanya akan menampilkan **sepenggal** dari keseluruhan kode yang ada pada *file* `.java` yang dimaksud. Hal tersebut menimbang kepentingan kode yang ingin dilampirkan guna mempertahankan penyampaian makna dan konsep secara efektif dan efisien. Keseluruhan blok kode akan tetap ditaruhkan di bagian yang berbeda.

1. Data: Vector3D & Triangle

a. Vector3D.java

```
// Vector3D.java
                                        ⋮


public Vector3D crossVector3d(Vector3D object) {
    Vector3D result = new Vector3D(
    (this.y * object.z  -  this.z * object.y),
    (this.z * object.x  -  this.x * object.z),
    (this.x * object.y  -  this.y * object.x)
    );

    return result;
    }

public double dotVector3D(Vector3D object) {
    double result = (this.x * object.x) + (this.y * object.y)
                                        + (this.z * object.z);
    return result;
    }


                                        ⋮
```

`Vector3D.java` merupakan *file* yang didedikasikan untuk mengimplementasikan operasi vektor yang mendasar, seperti *Addition, Subtraction, Dot Product, Cross Product,* dan *Normalization*. Operasi tersebut akan sangat krusial (karena juga mendasar) karena akan dipakai di dalam proses *intersection* yang telah dijelaskan pada bagian sebelumnya.

b. Triangle.java

```
// Triangle.java
                                        ⋮


public Triangle(Vector3D v0, Vector3D v1, Vector3D v2) {
                                        ⋮
    Vector3D edge1 = v1.subVector3d(v0);
    Vector3D edge2 = v2.subVector3d(v0);
    this.normal = (edge1.crossVector3d(edge2)).normalize();
    }

                                        ⋮
```

`Triangle.java` akan menyimpan tiga *vertex* (v1, v2, dan v0) dan menggunakan cross product dari dua sisi segitiga, lalu dinormalisasi. Hal ini dilakukan karena akan menjadi salah satu sumbu kandidat dalam algoritma **SAT** nantinya yang akan mendeteksi *intersection* antara kubus dan segitiga.

2. Input: Membaca.obj

```
// ObjectParser.java
                                        ⋮


public void parse(String filePath) {
        if (validateFile == false) { return; }
                                        ⋮
            String line;
            while ((line = br.readLine()) != null) {

                                        ⋮
                if (line.startsWith("v ")) {
                    parseVertex(line);
                } else if (line.startsWith("f ")) {
                    faceLines.add(line);
                }
                                        ⋮

            }
    }

                                        ⋮
```

`ObjectParser.java` akan membaca tiap baris dari *file* yang akan diberikan (valid jika `.obj`). Sebuah baris yang diawali dengan '`v`' itu menandakan informasi sebuah *vertex*. Sebuah baris yang diawali dengan '`f`' itu menandakan dirinya adalah informasi sebuah *face*. Setiap *vertex* akan dibaca dulu dan

dimasukkan ke dalam *list*, baru dilanjutkan dengan *face*. Hal tersebut mengkonsolidasikan *face* mereferensikan *vertex* by *index*.

Setiap *face* nanti akan dikonversi menjadi sebuah objek `Triangle`. Segitiga yang dinilai degeneratif (`getNormal().length()` mendekati 0 dengan *threshold* `1e-10`), akan langsung dibuang karena tidak dapat merepresentasikan permukaan yang sesuai.

3. Node Kubus: OctreeNode.java

```
// OctreeNode.java
                                        ⋮


public Vector3D getChildCenter(int octant) {
    double offset = this.halfSize / 2.0;

    double cx = this.center.getX() - offset;
    if ((octant & 1) != 0) cx = this.center.getX() + offset;

    double cy = this.center.getY() - offset;
    if ((octant & 2) != 0) cy = this.center.getY() + offset;

    double cz = this.center.getZ() - offset;
    if ((octant & 4) != 0) cz = this.center.getZ() + offset;

    return new Vector3D(cx, cy, cz);
}



                                        ⋮
```

Setiap node akan direpresentasikan dengan `center` & `halfSize` yang dimana akan mendefinisikan sebuah kubus secara penuh. Hal tersebut tersendiri dikarenakan setiap balok yang terbentuk akan berupa **kubus**. Dengan hanya mengetahui dua nilai tersebut, semua 8 sudut kubus bisa langsung dihitung dengan menambah atau mengurangi `halfSize` ke setiap koordinat x, y, z dari `center`.

4. SAT: IntersectionTest.java

```
// IntersectionTest.java
                                        ⋮

public static boolean intersects(OctreeNode octreeNode, Triangle triangle) {
    for (Vector3D boxAxes : getBoxAxes()) {
        // Sumbu X, Y, Z kubus
        boolean isOverlapsBoxAxes = overlaps(projectBoxAxes,
                                             projectTriangleAxes);
        if (isOverlapsBoxAxes == false) { return false; }

        // Sumbu Normal Segitiga
        boolean isOverlapsNormalTriangle = overlaps(projectBoxNormalTriangle,
                                            projectTriangleNormalTriangle);
        if (isOverlapsNormalTriangle == false) { return false; }

        // Sumbu Cross Product
        for (Vector3D crossAxes : getCrossAxes(triangle)) {
            if (crossAxes.length() < 1e-4) { continue; }
            boolean isOverlapsCrossAxes = overlaps(projectBoxCrossAxes,
                                                   projectTriangleCrossAxes);
            if (isOverlapsCrossAxes == false) { return false; }
        }

    return true;
}

                                        ⋮
```

Akan ada pengecekan pada 13 sumbu, yaitu 3 sumbu kotak, 1 normal segitiga, dan 9 dari *cross product*. Jika setiap dari 13 sumbu yang tadi tersebut proyeksinya akan *intersect*, maka dapat diasumsikan kesimpulannya kubus dan segitiga akan berpotongan. Sebagai catatan, sumbu *cross product* yang panjangnya mendekati nol (dengan threshold `1e-4`) dilewati karena tidak menghasilkan sumbu pemisah yang sesuai.

5. Divide & Conquer: Octree.java

```
// Octree.java
                                        ⋮

public void buildLeaf(OctreeNode octreeNode, int depth, List<Triangle>
triangles) {
    if (depth == maxDepth) {
        octreeNode.setLeaf(true);
        leaf.add(octreeNode);
        return;
    }
```

```
    for (int i = 0; i < 8; i++) {
        OctreeNode childNode = new OctreeNode(childeCenter, childHalfSize);
        List<Triangle> relevantTriangles = getRelevantTriangles(childNode,
                                                                triangles);

        if (relevantTriangles.isEmpty()) {
            statistics.recordPruned(depth + 1);
            octreeNode.setChild(i, childNode);
            buildLeaf(childNode, depth + 1, relevantTriangles);
        } else {
            octreeNode.setChild(i, childNode);
            buildLeaf(childNode, depth + 1, relevantTriangles);
        }
    }
}

                                        ⋮
```

`buildLeaf` merupakan sebuah fungsi rekursif yang akan membagi setiap *node* menjadi 8 oktan. Setiap dari oktan akan dicek dahulu apakah ada *intersect*. Jika tidak ada, oktan akan dibuang dan tidak diproses lebih lanjut ke rekursi selanjutnya. Jika sebaliknya, oktan akan diteruskan dan dipecahkan kembali lagi menjadi 8 oktan lebih kecil.

Hal tersebut akan membantu program berjalan dengan lebih efisien menimbang dengan melihat oktan yang dinilai relevan dengan `IntersectionTest.java`.

6. Output: ObjectWriter.java

```
// ObjectWriter.java

                                        ⋮
public String write(List<OctreeNode> leafNodes, String inputPath, int
maxDepth) throws IOException {
    String outputPath = generateOutputPath(inputPath);
    File outputFile = new File(outputPath);
    File parentDir = outputFile.getParentFile();

    try (FileWriter fw = new FileWriter(outputFile); BufferedWriter bw = new
BufferedWriter(fw)) {
        for (OctreeNode leafNode : leafNodes) {
            writeVertices(bw, leafNode);  // 8 vertex per voxel
        }

        for (int i = 0; i < leafNodes.size(); i++) {
            writeFaces(bw, (i * 8) + 1); // 12 face per voxel
        }
    }
```

```
    return outputPath;
}
```

⋮

Setiap *leaf node* yang dihasilkan (node "terdalam" alias hasil) akan dikonversi menjadi sebuah kubus dengan 8 vertex dan 12 segitiga (Ingat! 6 sisi berbentuk kotak x 2 segitiga). Setiap *vertex* akan ditulis terlebih dahulu yang dilanjut dengan seluruh *face* yang ada. Index *face* akan dihitung dengan *offset* $(i \cdot 8) + 1$ dikarenakan format `.obj` tersendiri yang menggunakan *indexing* berbasis 1 serta tiap *voxel*-nya akan memiliki 8 *vertex* (1 Node 8 Oktan konsepnya).

Keseluruhan kode untuk implementasi dari solusi yang dibawakan oleh kami terkait permasalahan *voxelization* akan ada di bawah ini. Untuk bagian ini, tidak ada penjelasan yang mendalam. Semua hal tersebut sudah dirangkum pada bagian di atas.

```
// Vector3D.java
public class Vector3D {
    private double x;
    private double y;
    private double z;

    public Vector3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    // Getter
    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    public double getZ() {
        return z;
    }

    // Operations
    public Vector3D addVector3d(Vector3D object) {
        //                                      X                   Y                   Z
        Vector3D result = new Vector3D( (this.x + object.x), (this.y + object.y), (this.z +
object.z) ); // Penamahan Matrix
        return result;
    }

    public Vector3D subVector3d(Vector3D object) {
        //                                      X                   Y                   Z
```

```
        Vector3D result = new Vector3D( (this.x - object.x), (this.y - object.y), (this.z -
object.z) ); // Pengurangan Matrix
        return result;
    }

    public Vector3D scalarVector3D (double num) {
        //                                  X               Y               Z
        Vector3D result = new Vector3D( (num * this.x), (num * this.y), (num * this.z) ); //
Scalar Matrix
        return result;
    }

    public Vector3D crossVector3d (Vector3D object) {
        /*
        |  i   j   k  |
        | a1  a2  a3  | => a x b = <(a2*b3 - a3*b2), (a3*b1 - a1*b3), (a1*b2 - a2*b1)>
        | b1  b2  b3  |
        */
        Vector3D result = new Vector3D( (this.y * object.z  -  this.z * object.y), (this.z *
object.x  -  this.x * object.z), (this.x * object.y  -  this.y * object.x));
        return result;
    }

    public double dotVector3D (Vector3D object) {
        /*
        a = [a1, a2, b3]
                            ==> a • b = (a1 * b1) + (a2 * b2) + (a3 * b3)
        b = [b1, b2, b3]
        */
        double result =  (this.x * object.x) + (this.y * object.y) + (this.z * object.z);
        return result;
    }

    public double length() {
        // Length     =      sqrt(         x^2          +           y^2         +           z^2
)
        double result = Math.sqrt(Math.pow(this.x, 2) + Math.pow(this.y, 2) + Math.pow(this.z,
2));
        return result;
    }

    // Normalization Matrix
    public Vector3D normalize() {
        double length = length();
        double normalized_X;
        double normalized_Y;
        double normalized_Z;

        if (length > 0) { // Pembagian untuk Sum(X, Y, Z) = 1
            normalized_X = x/length;
            normalized_Y = y/length;
            normalized_Z = z/length;
        } else {
            normalized_X = 0;
            normalized_Y = 0;
            normalized_Z = 0;
        }

        Vector3D result = new Vector3D(normalized_X, normalized_Y, normalized_Z);
        return result;
    }
}
```

```java
// Triangle.java
public class Triangle {
    private Vector3D v0;
    private Vector3D v1;
    private Vector3D v2;
    private Vector3D normal;

    public Triangle(Vector3D v0, Vector3D v1, Vector3D v2) {
        this.v0 = v0;
        this.v1 = v1;
        this.v2 = v2;

        Vector3D edge1 = v1.subVector3d(v0);
        Vector3D edge2 = v2.subVector3d(v0);
        this.normal = (edge1.crossVector3d(edge2)).normalize();
    }

    // Getter
    public Vector3D getV0() {
        return v0;
    }
    public Vector3D getV1() {
        return v1;
    }
    public Vector3D getV2() {
        return v2;
    }

    public Vector3D getNormal() {
        return normal;
    }
}
```

```java
// OctreeNode.java
public class OctreeNode {
    private Vector3D center;
    private double halfSize;
    private OctreeNode[] childrenNodes;
    private boolean isLeaf;
    private boolean isSurface;

    public OctreeNode(Vector3D center, double halfSize) {
        this.center = center;
        this.halfSize = halfSize;
        this.childrenNodes = new OctreeNode[8];
        this.isLeaf = false;
        this.isSurface = false;
    }

    // Getter
    public Vector3D getCenter() {
        return center;
    }
    public double getHalfSize() {
        return halfSize;
    }
    public OctreeNode[] getChildren() {
```

```
        return childrenNodes;
    }
    public OctreeNode getChild(int index) {
        if (index >= 0 && index <= 7) {
            return this.childrenNodes[index];
        } else {
            throw new IllegalArgumentException("Index: 0--7. Current index: " + index);
        }
    }

    public boolean isLeaf() {
        return isLeaf;
    }
    public boolean isSurface() {
        return isSurface;
    }

    // Setter
    public void setChild (int index, OctreeNode node) {
        if (index < 0 || index > 7) {
            throw new IllegalArgumentException("Index: 0--7. Current index: " + index);
        }
        if (this.childrenNodes[index] != null) {
            throw new IllegalArgumentException("Leaf node ke-"+ index +" sudah ada!");
        }
        this.childrenNodes[index] = node;
    }

    public void setLeaf(boolean value) {
        this.isLeaf = value;
    }

    public void setSurface(boolean value) {
        this.isSurface = value;
    }


    // Operations

    // Center untuk patokan perhitunan corners
    public Vector3D getChildCenter(int octant) {
        double offset = this.halfSize/2.0;

        // X coords
        double cx = this.center.getX();
        if ((octant & 1) == 0) {
            cx -= offset;
        } else {
            cx += offset;
        }

        // Y coords
        double cy =this.center.getY();
        if ((octant & 2) == 0) {
            cy -= offset;
        } else {
            cy += offset;
        }

        // Z coords
        double cz = this.center.getZ();
        if ((octant & 4) == 0) {
            cz -= offset;
```

```
        } else {
            cz += offset;
        }

        Vector3D result = new Vector3D(cx, cy, cz);
        return result;
    }

    // Half Size sebagai ukuran menuju corner dari center
    public double getChildHalfSize() {
        double result = this.halfSize/2.0;
        return result;
    }

    // Menghitung koordinat setiap corner
    public Vector3D[] get8Corners() {
        Vector3D[] corners = new Vector3D[8];

        // Menghitung koordinat setiap corner dari kubus.
        for (int i = 0; i < 8; i++) {
            double cx = this.center.getX();
            if ((i & 1) == 0) {
                cx -= this.halfSize; // left side
            } else {
                cx += this.halfSize; // right side
            }

            double cy = this.center.getY();
            if ((i & 2) == 0) {
                cy -= this.halfSize; // bottom side
            } else {
                cy += this.halfSize; // top side
            }


            double cz = this.center.getZ();
            if ((i & 4) == 0){
                cz -= this.halfSize; // back side
            } else {
                cz += this.halfSize; // front side
            }

            Vector3D corner = new Vector3D(cx, cy, cz);
            corners[i] = corner;
        }

        return corners;
    }
}
```

```
// Statistics.java
public class Statistics {
    public final int[] nodeCountPerDepth;
    public final int[] prunedCountPerDepth;
    public final int maxDepth;
    public int totalVoxels;
    public long startTime;
    public long endTime;
    public String outputPath;

    public Statistics(int maxDepth) {
        this.maxDepth = maxDepth;
```

```java
        this.nodeCountPerDepth = new int[maxDepth + 1];
        this.prunedCountPerDepth = new int[maxDepth + 1];
        this.totalVoxels = 0;
        this.startTime = 0;
        this.endTime = 0;
        this.outputPath = null;
    }

    // Getter
    public long getProgramTime() {
        return (this.endTime-this.startTime);
    }

    // Setter
    public void startTimer() {
        this.startTime =  System.currentTimeMillis();
    }

    public void stopTimer() {
        this.endTime = System.currentTimeMillis();
    }

    public void recordNode(int depth) {
        nodeCountPerDepth[depth]++;
    }

    public void recordPruned(int depth) {
        prunedCountPerDepth[depth]++;
    }

    public void recordVoxel() {
        totalVoxels++;
    }
}
```

```java
// IntersectionTest.java
import java.util.List;
import java.util.Vector;

public class IntersectionTest {
    private static final double EPSILON = 1e-6;

    // Getter

    // 1. Getter Box (X, Y, Z) -- 3 Sumbu
    public static Vector3D[] getBoxAxes() {
        Vector3D[] boxAxes = {new Vector3D(1, 0, 0),
                              new Vector3D(0, 1, 0),
                              new Vector3D(0, 0, 1)
                             };
        return boxAxes;
    }

    // 2. Getter Triangle Normalized -- 1 Sumbu
    public static Vector3D getTriangleNormalAxis(Triangle triangle) {
        return triangle.getNormal();
    }
    // 3. Getter Cross Product -- 9 Sumbu
    public static Vector3D[] getCrossAxes(Triangle triangle) {
        Vector3D t_v0 = triangle.getV0();
        Vector3D t_v1 = triangle.getV1();
        Vector3D t_v2 = triangle.getV2();
```

```
        Vector3D edge0 = t_v1.subVector3d(t_v0); // A ke B.
        Vector3D edge1 = t_v2.subVector3d(t_v1); // B ke C.
        Vector3D edge2 = t_v0.subVector3d(t_v2); // C ke A.

        Vector3D[] edges = {edge0, edge1, edge2};
        Vector3D[] boxAxes = getBoxAxes();
        Vector3D[] crossAxes = new Vector3D[9];

        int idx = 0;
        for (int i = 0; i < edges.length; i++) {
            for (int j = 0; j < boxAxes.length; j++) {
                crossAxes[idx++] = edges[i].crossVector3d(boxAxes[j]);
            }
        }
        return crossAxes;
    }

    // Memproyeksi setiap sudut untuk menentukan bayangan kubus (melihat Min-Max)
    public static double[] projectBox(Vector3D axis, Vector3D[] corners) {
        double min = axis.dotVector3D(corners[0]);
        double max = min;

        for (int i = 1; i < 8; i++) {
            double val = axis.dotVector3D(corners[i]);
            if (val<min) {
                min = val;
            }
            if (val>max) {
                max = val;
            }
        }
        double[] result = {min, max};
        return result;
    }

    // Memproyeksikan setiap sudut segitiga untuk menentukan bayangannya (melihat Min-Max)
    public static double[] projectTriangle(Vector3D axis, Vector3D[] vertices) {
        double min = axis.dotVector3D(vertices[0]);
        double max = min;
        for (int i = 1; i < 3; i++){
            double val = axis.dotVector3D(vertices[i]);
            if (val<min) {
                min = val;
            }
            if (val>max) {
                max = val;
            }
        }
        double[] result = {min, max};
        return result;
    }

    // Mengecek apakah overlaping
    public static boolean overlaps(double[] boxProject, double[] triangleProject) {
        boolean boxOverlapsTriangle;

        // Kubus akan overlap jika bayangannya sama atau lebih besar dibanding Segitiga.
        if ( (boxProject[1]+EPSILON) >= triangleProject[0]) {
            boxOverlapsTriangle = true;
        } else {
            boxOverlapsTriangle = false;
        }
```

```
        // Segitiga akan overlap bayangannya sama atau lebih besar dibanding Kubus
        boolean triangleOverlapsBox;
        if ( (triangleProject[1]+EPSILON) >= boxProject[0]) {
            triangleOverlapsBox = true;
        } else {
            triangleOverlapsBox = false;
        }

        // Overlaps hanya berlaku kalau keduanya true saja
        if ( (boxOverlapsTriangle == true) && (triangleOverlapsBox == true) ) {
            return true;
        } else {
            return false;
        }
    }

    // Separating Axis Thoerem, untuk mengecek apakah ada yang beririsan
    public static boolean intersects(OctreeNode octreeNode, Triangle triangle) {
        Vector3D[] corners = octreeNode.get8Corners();

        Vector3D t_v0 = triangle.getV0();
        Vector3D t_v1 = triangle.getV1();
        Vector3D t_v2 = triangle.getV2();
        Vector3D[] vertices = new Vector3D[]{t_v0, t_v1, t_v2};

        // Cek 3 sumbu koordinat
        for (Vector3D boxAxes : getBoxAxes()) {
            double[] projectBoxAxes = projectBox(boxAxes, corners);
            double[] projectTriangleAxes = projectTriangle(boxAxes, vertices);

            boolean isOverlapsBoxAxes = overlaps(projectBoxAxes, projectTriangleAxes);
            if (isOverlapsBoxAxes == false) {
                return false;
            }
        }
        // Cek normal triangle
        Vector3D triangleNormalAxis = getTriangleNormalAxis(triangle);
        double[] projectBoxNormalTriangle = projectBox(triangleNormalAxis, corners);
        double[] projectTriangleNormalTriangle = projectTriangle(triangleNormalAxis, vertices);

        boolean isOverlapsNormalTriangle = overlaps(projectBoxNormalTriangle,
projectTriangleNormalTriangle);
        if (isOverlapsNormalTriangle == false) {
            return false;
        }

        // Cek 9 Cross Product
        for (Vector3D crossAxes : getCrossAxes(triangle)) {
            if (crossAxes.length() < 1e-4) { // Don't over straight banget
                continue;
            }
            double[] projectBoxCrossAxes = projectBox(crossAxes, corners);
            double[] projectTriangleCrossAxes = projectTriangle(crossAxes, vertices);

            boolean isOverlapsCrossAxes = overlaps(projectBoxCrossAxes,
projectTriangleCrossAxes);
            if (isOverlapsCrossAxes == false) {
                return false;
            }
        }

        return true;
    }
```

```
    public static boolean anyIntersects(OctreeNode octreeNode, List<Triangle> triangles) {
        for (Triangle triangle : triangles) {
            if (intersects(octreeNode, triangle)) {
                return true;
            }
        }
        return false;
    }
}
```

```
// ObjectParser.java
import java.io.*;
import java.util.*;

public class ObjectParser {
    private List<Vector3D> vertices = new ArrayList<>();
    private List<Triangle> triangles = new ArrayList<>();
    private int lineNumber;

    // Getter
    public List<Vector3D> getVertices() {
        return vertices;
    }

    public List<Triangle> getTriangles() {
        return triangles;
    }

    public boolean validateFile(String filePath) {
        // 1. Null Validation
        if (filePath == null || filePath.trim().isEmpty()) {
            System.err.println("Path file NULL!");
            return false;
        }

        // 2. .obj Validation
        if (!filePath.toLowerCase().endsWith(".obj")) {
            String ext;
            if (filePath.contains(".")) {
                ext = filePath.substring(filePath.lastIndexOf('.'));
            } else {
                ext = "Tidak ada file!";
            }
            System.err.println("Hanya .obj yang valid. Input: " + ext);
            return false;
        }

        File file = new File(filePath);

        // 3. File Validation
        if (!file.exists()) {
            System.err.println("Tidak ada file di " + filePath);
            return false;
        }

        if (!file.isFile()) {
            System.err.println("Berikan File, bukan Folder");
            return false;
        }

        if (file.length() == 0) {
```

```
            System.err.println("File kosong!");
            return false;
        }

        return true;
    }

    public void parse(String filePath) {
        // File Validation
        boolean validateFile = validateFile(filePath);

        if (validateFile == false) {
            return;
        }

        vertices.clear();
        triangles.clear();
        lineNumber = 0;
        List<String> faceLines = new ArrayList<>();

        // Parsing .obj file
        try (FileReader fr = new FileReader(filePath); BufferedReader br = new
BufferedReader(fr)) {
            String line;
            while ((line = br.readLine()) != null) {
                lineNumber++;
                line = line.trim();
                if (line.isEmpty()) {
                    continue;
                }

                // Mengecek jenis informasi
                if (line.startsWith("v ")) {
                    parseVertex(line);
                } else if (line.startsWith("f ")){
                    faceLines.add(line);
                }
            }
        } catch (IOException e) {
            throw new RuntimeException("File gagal, Baris: " + lineNumber + ": " +
e.getMessage());
        }

        for (String faceLine : faceLines) {
            Triangle triangle = parseFace(faceLine);
            if (triangle != null) {
                triangles.add(triangle);
            }
        }

        if (vertices.size() < 3) {
            throw new IllegalStateException("Minimal 3 vertex!");
        }
        if (triangles.isEmpty()) {
            throw new IllegalStateException("Tidak ada segitiga");
        }

    }

    private void parseVertex(String line) {
        String[] lineSplits = line.split("\\s+");

        if (lineSplits.length != 4) { // Diasumsikan input akan hanya ada 4 kolom (v 1 2 3)
```

```
            throw new IllegalArgumentException("Format: v v_X v_Y v_Z");
        }

        try {
            // Format input dapat diasumsikan ([Type] X Y Z)
            double coords_X = Double.parseDouble(lineSplits[1]);
            double coords_Y = Double.parseDouble(lineSplits[2]);
            double coords_Z = Double.parseDouble(lineSplits[3]);

            if (Double.isNaN(coords_X) || Double.isNaN(coords_Y) || Double.isNaN(coords_Z) ||
Double.isInfinite(coords_X) || Double.isInfinite(coords_Y) || Double.isInfinite(coords_Z)) {
                throw new IllegalArgumentException("Informasi tidak valid " + lineNumber);
            }
            vertices.add(new Vector3D(coords_X, coords_Y, coords_Z));
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException(e.getMessage());
        }
    }

    private Triangle parseFace(String line) {
        String[] lineSplits = line.split("\\s+");

        if (lineSplits.length != 4) { // Diasumsikan input akan hanya ada 4 kolom (f 1 2 3)
            throw new IllegalArgumentException("Format: f f_X f_Y f_Z");
        }

        int[] indices = new int[3];
        for (int i = 0; i < 3 ; i++) {
            String lineSplit = lineSplits[i + 1];
            String indexString = lineSplit.split("/")[0];
            try {
                indices[i] = Integer.parseInt(indexString)- 1;
            } catch (NumberFormatException e) {
                throw new IllegalArgumentException(e.getMessage());
            }


            if (indices[i] < 0 || (indices[i] >= vertices.size()) ) {
                throw new IllegalArgumentException();
            }
        }

        Vector3D vector_A = vertices.get(indices[0]);
        Vector3D vector_B = vertices.get(indices[1]);
        Vector3D vector_C = vertices.get(indices[2]);
        Triangle triangle = new Triangle(vector_A, vector_B, vector_C);
        if (triangle.getNormal().length() < 1e-10) {
            System.err.println("Segitiga degeneratif! Baris: " + lineNumber);
            return null;
        }

        return triangle;
    }
}
```

```
import java.util.ArrayList;
import java.util.List;

public class Octree {
    private OctreeNode octreeNode;
    private int maxDepth;
```

```
    private Vector3D center;
    private double halfSize;
    private List<OctreeNode> leaf;
    private Statistics statistics;

    public Octree(int maxDepth) {
        this.maxDepth = maxDepth;
        this.leaf = new ArrayList<>();
    }

    // Getter
    public OctreeNode getOctreeNode() {
        return this.octreeNode;
    }

    public List<OctreeNode> getLeaf() {
        return this.leaf;
    }

    public Statistics getStatistics() {
        return statistics;
    }

    public void buildBox(List<Triangle> triangles) {
        // Instansiasi setiap variabel dengan MAX_VALUE biar bisa menerima setiap input di
iterasi pertama.
        double xMin = Double.MAX_VALUE;
        double xMax = -Double.MAX_VALUE;
        double yMin = Double.MAX_VALUE;
        double yMax = - Double.MAX_VALUE;
        double zMin = Double.MAX_VALUE;
        double zMax = - Double.MAX_VALUE;

        // Menentukan kubus terkecil yang dapat membungkus seluruh objek.
        for (Triangle triangle : triangles) {
            Vector3D[] triangleVectors = new Vector3D[]{triangle.getV0(), triangle.getV1(),
triangle.getV2()};

            for (Vector3D triangleVector : triangleVectors) {
                double xVal = triangleVector.getX();
                double yVal = triangleVector.getY();
                double zVal = triangleVector.getZ();

                // Min harus lebih kecil, Max harus lebih besar
                if (xVal < xMin) {
                    xMin = xVal;
                }
                if (xVal > xMax) {
                    xMax = xVal;
                }
                if (yVal < yMin) {
                    yMin = yVal;
                }
                if (yVal > yMax) {
                    yMax = yVal;
                }
                if (zVal < zMin) {
                    zMin = zVal;
                }
                if (zVal > zMax) {
                    zMax = zVal;
                }
            }
```

```
        }
        // Menyiapkan variabel untuk kubus baru dengan Center & HalfSize!
        double xCenter = (xMin + xMax)/2.0;
        double yCenter = (yMin + yMax)/2.0;
        double zCenter = (zMin + zMax)/2.0;
        this.center = new Vector3D(xCenter, yCenter, zCenter);

        double xRange = xMax-xMin;
        double yRange = yMax-yMin;
        double zRange = zMax-zMin;
        double maxRange = Math.max(xRange, Math.max(yRange, zRange));
        this.halfSize = (maxRange/2.0) * 1.01;
    }

    // Membuat kubus tersebut
    public void buildOctreeNode() {
        this.octreeNode = new OctreeNode(this.center, this.halfSize);
        this.octreeNode.setSurface(true);
        this.statistics = new Statistics(maxDepth);
    }

    // Untuk efisiensi, akan menggunakan filter saat menyentuh kubus saja (Intersects)
    public List<Triangle> getRelevantTriangles(OctreeNode octreeNode, List<Triangle> triangles)
{
        List<Triangle> result = new ArrayList<>();
        for (Triangle triangle : triangles) {
            if (IntersectionTest.intersects(octreeNode, triangle)) {
                result.add(triangle);
            }
        }
        return result;
    }

    // Divide n Conquer
    public void buildLeaf(OctreeNode octreeNode, int depth, List<Triangle> triangles) {
        // IF sudah ujung, END.
        if (depth == maxDepth) {
            octreeNode.setLeaf(true);
            leaf.add(octreeNode);
            statistics.recordNode(depth);
            statistics.recordVoxel();
            return;
        }

        statistics.recordNode(depth);

        // Iterasi untuk Divide n Conquer
        for (int i = 0; i < 8; i++) {
            Vector3D childeCenter = octreeNode.getChildCenter(i);
            double childHalfSize = octreeNode.getChildHalfSize();
            OctreeNode childNode = new OctreeNode(childeCenter, childHalfSize);

            List<Triangle> relevantTriangles = getRelevantTriangles(childNode, triangles);

            if (relevantTriangles.isEmpty()) {
                statistics.recordPruned(depth+1);
            } else { // Divide kalau ada segitiga yang beririsan dengan i'th child.
                childNode.setSurface(true);
                octreeNode.setChild(i, childNode);
                buildLeaf(childNode, depth+1, relevantTriangles);
            }
        }
    }
```

```
    public void collectLeaf(OctreeNode octreeNode, List<OctreeNode> leaf) {
        if (octreeNode.isLeaf()) {
            leaf.add(octreeNode);
            return;
        }
        for (int i=0; i<8; i++) {
            OctreeNode childNode = octreeNode.getChild(i);
            if (childNode != null) {
                collectLeaf(childNode, leaf);
            }
        }
    }
}
```

```
// VoxelGenerator.java
import java.util.ArrayList;
import java.util.List;

public class VoxelGenerator {
    private List<double[]> verticesList = new ArrayList<>();
    private List<int[]> facesList = new ArrayList<>();
    private int count = 0;

    // Getter
    public List<double[]> getVerticesList() {
        return verticesList;
    }

    public List<int[]> getFacesList() {
        return facesList;
    }

    private void generateVertices(Vector3D[] corners) {
        for (int i = 0; i < 8; i++) {
            double[] newVertice = new double[] {corners[i].getX(), corners[i].getY(),
corners[i].getZ()};
            verticesList.add(newVertice);
        }
    }

    // Membuat 12 Segitiga (2 Segitiga per Sisi dengan 6 total sisi di sebuah kubus)
    private void generateFaces(int offset){
        int[][] faces = { // List of faces, untuk tau segitiga yang mana aja
            // Kiri (X-)
            {0, 4, 2},
            {2, 4, 6},

            // Kanan (X+)
            {1, 3, 5},
            {3, 7, 5},

            // Bawah (Y-)
            {0, 1, 4},
            {1, 5, 4},

            // Atas (Y+)
            {2, 6, 3},
            {3, 6, 7},

            // Belakang (Z-)
            {0, 2, 1},
```

```
            {1, 2, 3},

            // Depan (Z+)
            {4, 5, 7},
            {4, 7, 6}
        };

        for (int[] face : faces) {
            int[] newFace = new int[]{(face[0]+offset + 1), (face[1]+offset +1),
(face[2]+offset + 1)};
            facesList.add(newFace);
        }
    }

    public void generate(List<OctreeNode> leaves) {
        verticesList.clear();
        facesList.clear();
        count = 0;

        for (OctreeNode leaf : leaves) { // Ngebuat kubus untuk setiap leaf node.
            int offset = count;
            Vector3D[] corners = leaf.get8Corners();
            generateVertices(corners);
            generateFaces(offset);

            count = count + 8;
        }
    }

}
```

```
// ObjectWriter.java
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.List;

public class ObjectWriter {
    private int[][] faces = {
        // Kiri (X-)
        {0, 4, 6},  {0, 6, 2},
        // Kanan (X+)
        {1, 3, 7},{1, 7, 5},

        // Bawah (Y-)
        {0, 1, 5}, {0, 5, 4},

        // Atas (Y+)
        {2, 6, 7}, {2, 7, 3},

        // Belakang (Z-)
        {0, 2, 3},{0, 3, 1},

        // Depan (Z+)
        {4, 5, 7},{4, 7, 6}
    };

    private void writeVertices(BufferedWriter bw, OctreeNode node) throws IOException {
        Vector3D[] corners =  node.get8Corners();
        for (int i = 0; i < 8; i++) {
            double v_X =  corners[i].getX();
            double v_Y = corners[i].getY();
```

```
            double v_Z = corners[i].getZ();

            bw.write(String.format("v %.6f %.6f %.6f", v_X, v_Y, v_Z));
            bw.newLine();
        }
    }

    private void writeFaces(BufferedWriter bw, int startVertexIndex) throws IOException {
        for (int[] face : faces) {
            int f_0 = face[0] + startVertexIndex;
            int f_1 =  face[1] + startVertexIndex;
            int f_2 =  face[2] + startVertexIndex;
            bw.write(String.format("f %d %d %d", f_0, f_1, f_2));
            bw.newLine();
        }
    }

    public String write(List<OctreeNode> leafNodes, String inputPath, int maxDepth) throws
IOException {
        String outputPath = generateOutputPath(inputPath);
        File outputFile = new File(outputPath);
        File parentDir = outputFile.getParentFile();

        if (parentDir != null && parentDir.exists() == false) {
            if (parentDir.mkdir() == false) {
                throw new IOException();
            }
        }

        try (FileWriter fw = new FileWriter(outputFile); BufferedWriter bw = new
BufferedWriter(fw)) {
            for (OctreeNode leafNode : leafNodes) {
                writeVertices(bw, leafNode);
            }

            for (int i = 0; i < leafNodes.size(); i++) {
                writeFaces(bw, (i * 8) + 1);
            }
        }
        return outputPath;
    }

    // Penulisan path
    public String generateOutputPath(String inputPath) {
        File file = new File(inputPath);
        String name = file.getName();

        int dotIndex = name.lastIndexOf('.');
        String base;
        if (dotIndex != -1) {
            base = name.substring(0, dotIndex);
        } else {
            base = name;
        }

        String parent = file.getParent();
        // output file format <file name>-voxelized.obj
        if (parent != null) {
            return parent + File.separator + base + "-voxelized.obj";
        } else {
            return base + "-voxelized.obj";
        }
    }
```

```
}
```

```
// Reporter.java

public class Repoerter {
    private final Statistics statistics;

    public Repoerter(Statistics statistics) {
        this.statistics = statistics;
    }

    private void printSeparator() {
        System.out.println("==================================");
    }

    private void printVoxel() {
        int voxels = statistics.totalVoxels;
        int vertices = voxels * 8;
        int faces = voxels * 12;
        System.out.printf("Voxel: %d%n", voxels);
        System.out.printf("Vertices: %d%n", vertices);
        System.out.printf("Faces: %d%n", faces);
    }

    private void printNode() {
        System.out.println("Node Octree");
        for (int i = 0; i < statistics.maxDepth; i++) {
            System.out.println((i+1) + " : " + statistics.nodeCountPerDepth[i+1]);
        }
    }

    private void printPrune() {
        System.out.println("Pruned Octree");
        for (int i = 0; i < statistics.maxDepth; i++) {
            System.out.println((i+1) + " : " + statistics.prunedCountPerDepth[i+1]);
        }
    }

    private void printTime() {
        System.out.printf("Start: %.3f | End: %.3f%n", statistics.startTime/1000.0,
statistics.endTime/1000.0);
        System.out.println("Waktu Eksekusi: " + statistics.getProgramTime()/1000.0);
    }

    public void report() {
        printSeparator();
        printVoxel();
        printSeparator();
        printNode();
        printSeparator();
        printPrune();
        printSeparator();
        printTime();
        printSeparator();
        System.out.println("Selesai reportnya");
    }
}
```

```
// main.java
import java.io.IOException;
import java.util.List;
```

```java
public class main {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.err.println("java -cp src main test/<.obj> <maxDepth>");
            System.exit(1);
        }

        String filePath = args[0];
        int maxDepth;

        try {
            maxDepth = Integer.parseInt(args[1]);
        } catch (NumberFormatException e) {
            System.err.println(e.getMessage());
            System.exit(1);
            return;
        }

        if (maxDepth <= 0) {
            System.err.println("maxDepth: 0--10");
            System.exit(1);
        }
        if (maxDepth > 10) {
            System.err.println("maxDepth: 0--10");
        }

        Statistics statistics;

        // Menyiapkan memproses input
        ObjectParser objectParser = new ObjectParser();
        objectParser.parse(filePath);
        List<Triangle> triangles = objectParser.getTriangles();

        // Menyiapkan OctreeNodes
        Octree octree = new Octree(maxDepth);
        octree.buildBox(triangles);
        octree.buildOctreeNode();

        statistics = octree.getStatistics();
        statistics.startTimer();

        // Membentuk leaf nodes (proses DnC fyi)
        octree.buildLeaf(octree.getOctreeNode(), 0, triangles);
        List<OctreeNode> leaves = octree.getLeaf();

        // ObjectWriter untuk output ditulis hasil DnC
        ObjectWriter objectWriter = new ObjectWriter();
        try {
            String output = objectWriter.write(leaves, filePath, maxDepth);
            statistics.outputPath = output;
        } catch (IOException e) {
            System.err.println(e.getMessage());
            System.exit(1);
        }

        statistics.stopTimer();
        Repoerter repoerter = new Repoerter(statistics);
        repoerter.report();
        System.exit(0);
    }
}
```

Analisis Kompleksitas

Kompleksitas akan melihat dari 4 fungsi utama, yaitu fungsi untuk *parsing*, fungsi untuk SAT *Intersection Test*, fungsi untuk membuat Node-Node dalam *Divide* & *Conquer*, dan fungsi untuk penulisan *output*.

A. Parsing

Kompleksitas untuk ini akan berfokus pada ObjectParser.java yang seharusnya akan memiliki kompleksitas $O(V + T)$, yang dimana variabel V untuk jumlah *vertex* dan variabel T untuk jumlah *triangle*. Hal tersebut kepikiran dikarenakan pada sebuah .obj *file* akan berisikan V baris informasi *vertex* dan T baris informasi *triangle*.

B. SAT Intersection Test

Untuk *function* terkait pengecekan *intersect* di antara *node* dan *triangle*, proses pengujian akan melihat 3 kotak, 1 normal segitiga, dan 9 *cross product* dengan total 13. Total tersebut akan selalu konstan di setiap *test case* ataupun iterasi lainnya. Setiap pengujian juga pasti akan hanya memberikan *output* proyeksi ke 8 sudut kubus dan 3 *vertex* segitiga. Sehingga, hal tersebut dapat disimpulkan kompleksitas $O(1)$ yang didasari oleh alasan nilai-nilai yang dimaksud tadi selalu konstan.

C. Divide & Conquer Leaf Node

Untuk *function* ini, sebenarnya terdapat 2 kasus yang perlu diperhatikan. Pertama, kompleksitas algoritma ini dengan memanfaatkan efisiensi dengan menggunakan *pruning*. Seperti yang sudah dijelaskan, konsep ini akan hanya meneruskan *node* jika ada segitiga yang berpotongan. Nanti, setiap T segitiga, pada setiap D *depth*, akan hanya menghasilkan n banyaknya *intersection test*($O(1)$). Melihat hal tersebut, kami dapat membentuk kesimpulan untuk kompleksitas pada kasus ini berupa $O(T \cdot D)$.

Kedua, terkait *worst case* yang dimana tidak melakukan pruning sama sekali. Hal tersebut akan membuat di setiap d *depth*, akan memiliki 8^d banyaknya *node* yang setiap *node*-nya akan mengecek lebih dalam sebanyak T *triangle*. Jadinya, nilai kompleksitasnya akan $O(T \cdot 8^D)$.

D. Output

Bagian ini akan meninjau untuk `ObjectWriter.java` yang akan mengiterasi seluruh *leaf node* sebanyak total *voxel*. Setiap *leaf node* akan menghasilkan 8 *vertex* dan 12 *face* secara konstan, sehingga tidka perlu dimasukkan ke dalam perhitungan. Maka, kompleksitas nya adalah $O(L)$

Maka dari itu, kompleksitas keseluruhan akan dapat disimpulkan dengan $O(T \cdot D + L)$. Dengan tentunya *worst case* tidak jauh seperti yang sudah dijelaskan, yaitu $O(T \cdot 8^D)$.

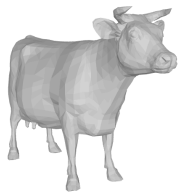
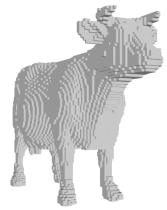
Pengujian *Test Case*

A. Pengujian 1: `cow.obj`

Input File

⋮

```
v 2.348182 -1.624595 0.000000
v 2.167167 -1.596326 0.000000
v 1.920432 -1.567396 0.000000
v 2.881063 -0.493146 -0.521968
v 2.736198 -0.346225 -0.659345
v 2.835437 -0.231101 -0.577890
v 2.986408 -0.413503 -0.428964
v 2.910769 -0.638444 -0.431191
v 2.825540 -0.567578 -0.552597
v 2.688131 -0.517509 -0.672975
v 2.647368 -0.611550 -0.669485
v 2.618307 -0.549092 -0.805491
v 2.993075 -0.708434 -0.342596
v 3.045715 -0.767786 -0.261741
v 3.065544 -0.788768 0.000000
v 3.107533 -0.691019 -0.264602
v 3.118124 -0.703925 0.000000
v 3.053144 -0.609681 -0.326931
f 3 4 2
f 1 2 4
f 5 6 3
f 4 3 6
f 6 5 7
f 7 8 6
f 8 9 6
f 10 4 9
f 6 9 4
f 11 1 10
f 4 10 1
```

Input Image 	**Output Image** 

Keterangan

```
C:\Users\Ardy\OneDrive - Institut Teknologi Bandung\Codingan\SEMESTER
6\Stima\Tugas_Kecil_Dua>java -cp src main test/cow.obj 7
=====================================
Voxel: 22343
Vertices: 178744
Faces: 268116
=====================================
Node Octree
1 : 8
2 : 20
3 : 86
4 : 339
5 : 1349
6 : 5502
7 : 22343
=====================================
Pruned Octree
1 : 0
2 : 44
3 : 74
4 : 349
5 : 1363
6 : 5290
7 : 21673
=====================================
Start: 1774680993.128 | End: 1774680994.199
Waktu Eksekusi: 1.071
=====================================
Selesai reportnya
```

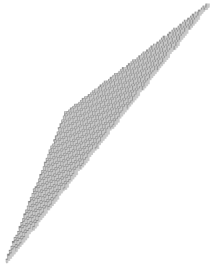
B. Pengujian 2: `line.obj`

Input File

```
v 2.229345 -0.992723 -0.862826
v 2.292449 -0.871852 -0.882400
v 2.410367 -0.777999 -0.841105
f 1 2 3
```

Input Image

Output Image



Keterangan

```
C:\Users\Ardy\OneDrive - Institut Teknologi Bandung\Codingan\SEMESTER
6\Stima\Tugas_Kecil_Dua>java -cp src main test/line.obj 7
=====================================
Voxel: 3763
Vertices: 30104
Faces: 45156
=====================================
Node Octree
1 : 7
2 : 15
3 : 33
4 : 92
5 : 285
6 : 1007
7 : 3763
=====================================
Pruned Octree
1 : 1
2 : 41
3 : 87
4 : 172
5 : 451
6 : 1273
7 : 4293
=====================================
Start: 1774681088.869 | End: 1774681089.339
Waktu Eksekusi: 0.47
=====================================
Selesai reportnya
```

C. Pengujian 3: `pumpkin.obj`

Input File

⋮

```
v -6.533880 31.363400 -86.093018
v -6.096960 32.236698 -87.062164
v -4.792930 30.981300 -134.378387
v -7.100460 30.430000 -134.722900
v -8.699420 30.741501 -134.141800
v -11.148100 29.617100 -134.453796
v -13.924900 29.668301 -133.587997
v -15.106500 27.610901 -135.006302
v -20.578400 25.695400 -134.798294
v -14.890300 29.193800 -85.873055
v -7.105960 32.634800 -87.696960
v -16.709900 27.799400 -133.729691
v -18.020300 27.246099 -133.444992
v -21.541599 25.314301 -134.672195
v -4.959950 32.541302 -132.252899
v -6.750670 32.398800 -132.476501
v -8.131280 31.791599 -133.177200
v -9.947160 30.646299 -133.954590
v -17.602699 29.331699 -131.410492
v -15.588000 29.067600 -133.448898
v -4.509000 3.664000 -78.843697
f 3971 3877 3788
f 2489 2608 2447
f 3686 3472 3473
f 2489 2533 2647
f 3686 3685 3472
f 3744 3788 3877
f 3685 3744 3472
f 3744 3685 3788
f 2409 2533 2489
f 3877 3743 3744
```

⋮

Input Image

Output Image

Keterangan

```
C:\Users\Ardy\OneDrive - Institut Teknologi Bandung\Codingan\SEMESTER
6\Stima\Tugas_Kecil_Dua>java -cp src main test/pumpkin.obj 7
=====================================
Voxel: 73273
Vertices: 586184
Faces: 879276
=====================================
Node Octree
1 : 8
2 : 56
3 : 277
4 : 1102
5 : 4538
6 : 18234
7 : 73273
=====================================
Pruned Octree
1 : 0
2 : 8
3 : 171
4 : 1114
5 : 4278
6 : 18070
7 : 72599
=====================================
Start: 1774681065.987 | End: 1774681068.541
Waktu Eksekusi: 2.554
=====================================
Selesai reportnya
```

D. Pengujian 4: `teapot.obj`

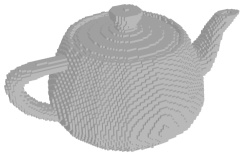
Input File

```
vn  -0.14474  0.925312  -0.350498
vn  -0.201474  0.850653  -0.485589
vn  6.16189e-09  0.971611  -0.236584
vn  -1.37744e-09  0.984907  -0.173084
vn  -2.79781e-10  0.925108  -0.379703
vn  -2.55497e-09  0.850068  -0.526673
vn  0.0902162  0.971775  -0.217978
vn  0.0659615  0.984977  -0.15959
vn  0.14474  0.925312  -0.350498
vn  0.201474  0.850653  -0.48559
vn  0.166631  0.971838  -0.166631
vn  0.121908  0.985026  -0.121908
vn  0.267668  0.925585  -0.267668
vn  0.371315  0.851029  -0.371315
vn  0.217978  0.971775  -0.0902162
vn  0.15959  0.984977  -0.0659615
vn  0.350498  0.925312  -0.14474
```

```
vn  0.48559  0.850653  -0.201474

f  7  6  1
f  1  2  7
f  8  7  2
f  2  3  8
f  9  8  3
f  3  4  9
f  10  9  4
f  4  5  10
f  12  11  6
f  6  7  12
f  13  12  7
f  7  8  13
f  14  13  8
```

⋮

Input Image	**Output Image**
	

Keterangan

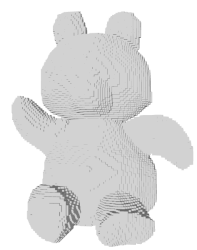
```
C:\Users\Ardy\OneDrive - Institut Teknologi Bandung\Codingan\SEMESTER
6\Stima\Tugas_Kecil_Dua>java -cp src main test/teapot.obj 7
Segitiga degeneratif! Baris: 2090
Segitiga degeneratif! Baris: 2090
Segitiga degeneratif! Baris: 2090
```

```
v 3.89555 -16.6003 -5.65267
v 7.1955 -17.2256 -2.10135
v 5.50509 -16.7471 -4.66901
v 6.13653 -15.9249 -5.25062
v 6.29974 -17.1925 -3.43905
v 6.44628 -14.1865 -6.82055
v 5.00347 -16.345 -5.4749
v 6.98854 -16.9314 -3.18964
f 3 4 2
f 8 9 7
f 8 7 6
f 10 9 8
f 14 13 4
f 7 16 15
f 9 17 16
f 9 16 7
f 21 22 20
f 21 20 19
f 23 24 22
f 23 22 21
f 25 26 23
```

⋮

Input Image

Output Image



Keterangan

```
C:\Users\Ardy\OneDrive - Institut Teknologi Bandung\Codingan\SEMESTER
6\Stima\Tugas_Kecil_Dua>java -cp src main test/teddybear.obj 7
===================================
Voxel: 42932
Vertices: 343456
Faces: 515184
===================================
Node Octree
1 : 8
2 : 34
3 : 142
4 : 642
5 : 2626
6 : 10672
7 : 42932
===================================
Pruned Octree
1 : 0
2 : 30
3 : 130
4 : 494
5 : 2510
6 : 10336
7 : 42444
===================================
Start: 1774681692.459 | End: 1774681693.915
Waktu Eksekusi: 1.456
===================================
Selesai reportnya
```

Repository

https://github.com/SArdyBramantyo/Tucil2_18223057_18223025

Pernyataan Kejujuran

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Stanislaus Ardy Bramantyo

Muhammad Azzam Robbani

Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✅	
2	Program berhasil dijalankan	✅	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✅	
4	Program informasi-informasi atau report	✅	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✅	
6	[Bonus] Program menggunakan *concurrency*		❌
7	[Bonus] Program dapat menampilkan visualisasi .obj		❌